

2026-08-01

The bar for the completed list is one thing: **would I defend this sentence for twenty hours against someone trying to break it**. Anything that fails that test is in section 2 or section 3, however much work went into it.

A result is only here if it is (a) measured rather than argued, (b) measured on data that passes `engine/quality.js` , and (c) still true when re-run today. Several things that were on this list a week ago have been removed for failing (c).

87% of the ladder store is unusable and we know exactly why. 15,973 stored games, ~2,057 clean. Bots, forfeits, partial brings and stubs, each counted separately in a funnel that anyone can re-run. 27 assertions in `tests/test-quality.js` .

Behavioural bot detection works on accounts that do not announce themselves. Team invariance: an account playing 50+ games without ever changing a slot is a script. It caught six accounts that appeared in **52.2%** of the games a name-only filter called clean. Carchdraw84172 played 459 games with one team, 367 of them in a single day.

Defensible because: the rule is stated, the evidence is per-account, and the false-positive check is published — no account with ≥ 50 games in the store has more than one distinct team, so the rule separates cleanly.

Known limit, stated: a bot playing under 50 games or varying its team is invisible. The right phrase is always "no bot detected", never "human".

31 of 31 scenarios within 2% of `@smogon/calc` , through the official Showdown engine pinned at commit `20ad99f` . Golden-master, re-run before every engine change.

Defensible because: it is validated against an independent implementation, not against itself, and the harness caught two of its own wiring bugs on first run.

600 seed-matched battles per policy — same teams, same seeds, same engine — against 1,905 clean ladder games:

	before	after	real humans
super effective	9.71%	14.91%	21.37%
moves that outright failed	9.68%	6.34%	2.47%
moves that hit an immune target	4.30%	2.92%	1.91%

Defensible because: the fit never saw the realism report — it was held back as the out-of-sample check — and the comparison is paired on seeds so team difficulty cancels.

What this does NOT claim: that MAG is good. See 2.1.

A challenger optimised for **winning** rather than for resembling humans, drawn from MAG's own 17 features, beats MAG **63.2%** — and beats MAG's predecessor **68.2%** where MAG itself manages **60.2%**.

Defensible because: the 68.2% is an independent evaluation the search never optimised against, on an opponent it was not tuned for. It is transitively better, not a counter.

RETRACTED 2026-08-04 — all three figures describe a 17-feature vector we no longer ship. They were computed on an engine 25 wire-fixes old and **before the quality filter existed**; `provenance.js` carried `data/exploitability.json` as its only `UNSAFE` artifact on exactly that ground. The re-run on the shipped 58 features is **void** — `data/policy-weights.json` was refitted at 22:15:24 UTC while the search was running and `engine/medicham2-browser.js` moved twice more afterwards — and its hill-climb accepted **1 of 24** steps, so it was uninformative regardless. See `docs/SEARCH.md` §R8. **The**

claim of this section may still be true; the evidence for it no longer exists. The general point that imitation and winning are different objectives stands on its own separate evidence (see `MAGNEMITE` in `docs/MODELS.md`), not on these three numbers.

And it is interpretable: the winning weights barely care about type effectiveness and care enormously about not wasting a turn and finishing what is already hurt.

Teams differ enormously between open- and closed-sheet play — **551.9 points** of total absolute species difference, Garchomp on 81.6% of open-sheet teams against 47.7% of closed. Behaviour given a board differs by **at most 1.49 points** on every measure.

Defensible because: both sides were measured with the same code, and the policy is conditional on the board — it never learns what to bring, and MEW draws teams from the ladder store regardless.

`engine/fit_policy.js` re-checks it on every refit by re-estimating on a reweighted sample.

The Bo3 **open-team-sheet** ladder is now pulled hourly into its own store. That format carries `Force Open Team Sheets`, so every game publishes all six sets — the only continuously-collected corpus where the choice set of a decision is known rather than guessed.

Defensible because: the format's ruleset was read from the simulator, the pull was tested live, and the store is separate — Bo3 is a different information regime and pooling it would corrupt every behavioural statistic.

Ability blocking (Levitate/Ground, Flash Fire/Fire, Good as Gold/status, Armor Tail/priority) is **read out of 14,933 recorded battles**, not written down — including inferring *which rule* each ability follows by testing candidate hypotheses and preferring the narrower on ties.

Defensible because: it was verified against clean-only data and every rule is identical; and because probing Showdown's handlers directly was tried first and **failed silently**, which is why the measured route was taken.

- `engine/provenance.js` — every artifact checked for staleness against the filter, its inputs, and the clean-game count. Found **28 unsafe to quote**.
- `engine/conformance.js` — S1–S13 as executable checks over 132 files. **S12 is now clean**.
- `engine/selftest.js` — 24 assertions on failures that are silent by nature.
- `build/build_pdfs.js` — every document has a current PDF, list derived.

Defensible because: each found real defects on its first run, including in the tool that found them.

It is **exploitable** (1.4), it still repeats Protect at roughly double the human rate, it wastes 6.34% of moves against a human 2.47%, and its one head-to-head win was against a baseline with a flaw it was purpose-built to punish. **It is a starting position, not a player.**

One confirmed promotion. The round robin that would establish whether the ladder climbs or goes in circles has never had enough games to resolve anything. Reported as a null.

o UNSAFE, 45 ok. Twenty-five were regenerated by re-running their own generators; the last was fixed by addressing what the gate was actually complaining about. `provenance.js --strict` is wired into the suite and green. This was named "the blocker on everything in section 3" and it is no longer blocking anything.

Two real bugs were underneath the staleness, neither of which waiving by name would have found: `illusion.json` carried a sound raw-store justification in its generator that never travelled to the artifact; `archetypes.json` reported **team sides** (2 per game) under the name `n_games`, so every share and silhouette in it had the wrong sample size on its label.

What this does NOT establish. `provenance.js` says so itself: it checks what artifacts DECLARE. o UNSAFE means nothing is misdeclared, not that every number is right. **14 are still flagged "possibly stale"**, and only re-running a generator can settle whether its output is correct.

Recorded so they are not quoted again:

model	state
PORY	its comparison artifact was trained without the filter; standing unknown , not weak
SLOWKING	mathematics sound, but the clean matrix has o decisive matchups — no usable input
GURU	o decisive matchups on clean data; descriptive only
JOLTEON	ties a coin; recommended for retirement
WAR	withdrawn — the signal was four bots playing one team 1,446 times
CHOMP-EV	honest null; brings do not separate from a coin
the 199,524-game self-play corpus	unusable — generated before megas worked, with duplicate Protects on one set
"the meta is rock-paper-scissors"	withdrawn; computed pre-filter, deleted

Self-play has none — we run both sides. Nothing in the current work teaches the bot to play against an unknown team, which is the defining difficulty of the closed-sheet ladder.

Items 3, 4 and 5 below have shipped. Recorded rather than deleted, because a roadmap that quietly drops its own entries cannot be audited.

- **"Give MAG a damage calculation, and a KO check both ways"** — DONE. `board.js` calls the real damage engine throughout; `koTarget`, `killIsRoll`, `diesBeforeMoving` and the switch-survival features all read it.
- **"Score both Pokémon together"** — BUILT AND MEASURED, and it **lost**: 42.0% [39.9, 44.3] over 1,934 paired games, 28.4% of decisive pairs, against its own zeroed control. The imitation fit is refuted; the idea is not. Now item 1 below.
- **"Switching"** — no longer a coin flip. Voluntary switches score through `switchFeatures`, the post-KO replacement is scored rather than rolled, and `entryHits = forced ? 0 : 1` separates the two cases.

A process failure that caused real damage, recorded so it is not repeated. On 2026-07-30 the model family was repeatedly mischaracterised in conversation — DODUO described as unbuilt when it had been built, wired, controlled and measured; MEDICHAM audited in its graveyard version; top-K pruning "proposed" when `fit_joint.js` already had it. Cause: two full days and ~40 commits of work existed in NO document. `MODELS.md` was last written 2026-07-28; every other living doc 2026-07-26. The only reason DODUO's verdict was findable at all is that its experiment happened on the one day `MODELS.md` was touched. **CLAUDE.md requires the docs to move in the same pass as the code. They did not.**

- **"Choice lock in the candidate set"** (was item 2) — DONE. The logit denominator no longer contains moves the human could not click. After the refit, six of eight switch features clear zero, and **switches now win the argmax** where `--switching` previously changed nothing.
- **The tag artifact reaches the feature vector** — `speedSwing` +0.983, `screenValue` +1.128, `healValue` +2.220, all clearing zero by wide margins. 53 → 56 features. This is the first feature addition since 3.28.0's four nulls to measure as real, and the difference is that these are CONDITIONS on mechanics with no prior representation rather than restatements.
- **ALAKAZAM job 2 (guess what they will do)** — built, off by default, needs a refit. The max in `incomingThreat` is now an expectation over P(their action).
- **A gate against wasted runs** — `engine/preflight.js`, 24 games, refuses a training run whose feature blocks receive no gradient. Two 1.5-hour runs were lost to exactly that before it existed.

- **DODUO is trainable for winning** (was item 1, the top of section 3). Wiring only, as the item predicted. `--joint` trains the 74-length vector on the game result; `--joint-only` freezes the 56 singles so the whole trust-region step reaches the 18 pair terms. Four separate faults would each have produced a plausible, wrong learning curve, and every one was caught by a check rather than by reading. **The head-to-head is running; no win rate is claimed.**
- **The trust region starved the pair block, structurally.** The first run moved it 4.7% while the singles moved 21.7% — single features appear in nearly every choice set, pair features are sparse, so they lose a shared budget by construction and more iterations scale both equally. `--joint-only` took it to **40.0%**, with three sign flips: `overkill` $-0.951 \rightarrow +0.980$, `focusFireKills` $-0.107 \rightarrow +1.094$, `partnerCoversMe` $-0.004 \rightarrow +0.639$, and `bothSameTarget` $0.031 \rightarrow 2.249$. The imitation fit penalised focus fire; training to win reverses it.
- `mew_farm` **exited 0 with every worker dead.** A farm that wrote zero games was indistinguishable, to anything checking the exit code, from a clean run. Found because the DODUO gate refused to start.
- **The speed-multiplier divergence is cross-checked** (systems audit R2, which the audit could not finish because `board.js` was off-limits mid-run). `tests/test-speed-multipliers.js`, 15 passed, compares behaviour as ratios and compares the SETS, so a new weather-speed ability in the dex cannot hide behind four passing constants.
- **A test that asserted a count.** `sanity_check.py` required Sun > 1000 teams; Sun is the third largest of eight styles. Now relative and mutation-tested. **96 passed, 0 failed.**
- **All twelve recommendations from the three 2026-07-31 reviews are closed.** Including the two that had been built but never reached a reader: the multiplicity correction (on disk since 14:04, in no reader-facing document) and the retired NMF justification, still live in `PUBLICATION.md` — the document that exists to be published — while the honest version sat in `SUMMARY.md`.
- **There is a rollback point.** `v3.31.0`, tagged and pushed, with the one known failure named in the tag message. Both reviews had recorded that nothing existed to roll back to.

Will played MAG in the real Showdown client for the first time — six games. **Eleven defects in about two hours**, more than the 47 test files had surfaced. Full evidence: `docs/FINDINGS-2026-08-01-live-play.md`.

Handoff: `docs/HANDOFF-2026-08-01.md`.

- **The bot was not delivering the model to the board.** `makeScoringPlayer(opts)` accepted an options object and never used it — the class reads its own second constructor argument — so `{greedy: true, switching: true}` configured nothing. MAG played its first real games *sampling* its moves instead of taking its best one. Fixed by merging factory options into constructor options.
- `joint: true` **silently disabled mega evolution.** `_withMega` had exactly one call site and the joint path returned around it. Both joint paths now go through it, and `test-wiring.js` re-asserts mega, aiming and open sheets **under** `--joint` — the broken cell was the intersection of two capabilities each tested alone. Fourth mega defect in this project.
- **Voluntary switching was reverted to off.** The bot had requested it since its first version, against `mew.js:135` 's measured verdict of a **10-point loss**. It only took effect once the options bug was fixed — an unmeasured change against a measured result.
- **Item Clause repair could never succeed.** It resampled while telling the sampler the answer had to be the colliding item, so the "rare" no-item fallback fired every time.
- Open team sheets are now accepted; failed team draws redraw instead of rejecting the challenge.

Nothing about MAG's playing strength was established. MAG went 1–2 in the three games where the policy was actually driving. Six games is not evidence in either direction.

o. REFIT — and it may rewrite item 1 below. `board.js` changed a feature's *meaning* under an unchanged *name*, so every shipped weight was fitted against the old definition. `board.js` is required by 40 files; the vector is read by 24. Nothing in the project detects this: `magnemite.js:297` compares feature names and `:299` compares vector length, and both pass when semantics drift. A guard — a stored hash of each feature's values over a fixed fixture board — is the cheapest high-leverage item on this list, and the only one that prevents a repeat rather than fixing an instance.

Why this comes before item 1, and may dissolve it. Item 1 rests on `spreadFreeBesideAlly` being priced at -5.054 , read as "*humans rarely click this, not this is bad.*" That feature fires only when `allyHit === 0`, and `allyHit` used `getEffectiveness >= 0`, which **returns 0 for an immunity**. So the feature's coverage was inverted at its most important case:

ally beside a spread move	did <code>spreadFreeBesideAlly</code> fire?	is it actually free?
Flying partner + Earthquake	no	yes — fully immune
Levitate partner + Earthquake	yes (ability path)	yes
merely <i>resisting</i> partner	yes	no — it still takes damage

The clearest free-spread case in VGC never fired once, while a case that costs real HP did. A weight fitted on that is a weight fitted on the wrong question, and **-5.054 may be an artifact rather than a preference**. Stated as a hypothesis; the refit is the test. If it survives the refit, item 1 stands exactly as written.

RESOLVED 2026-08-01 — it WAS an artifact, and of something else entirely **The immunity hypothesis above is refuted, and the conclusion it was reaching for is confirmed by a different cause.** Both fits were re-run. **The refit alone changed almost nothing.** `allyHit` moved from -0.0187 to -0.0126 (unweighted $-0.0937 \rightarrow -0.0875$) — about **0.2 standard errors**, i.e. noise. Measured rather than argued: the immunity fix changes **0.198% of candidates**, and `allyHit` fires on only 0.63% of them to begin with. So the honest reading of `allyHit` is the plain one — **humans do not avoid hitting their own partner much** — and it should be reported that way rather than re-explained. **What produced -4.986 was the FITTER, not the feature.** `fit_joint.js` matched a human's click by requiring the candidate's target to match, and `board.js` builds a spread candidate with `targetMon: null` because Earthquake is not aimed. **No spread click could ever match.** Measured over 400 corpus games: spread moves are **14.94% of all human move clicks**, 99.71% carry a recorded target, and **1,393 of 1,397 failed to match**. Because a joint turn needs both slots, the loss compounded — the fit discarded **57,486 of 82,483 joint turns** and estimated 74 weights from 30% of the data. The discarded 70% was not random: it was exactly the turns containing a spread move. `spreadFreeBesideAlly` therefore fired on 8.0% of the enumerated alternatives and **0 of 1,461 pairs a human actually chose**, and a conditional logit reads "always available, never chosen" as a large negative. `fit_policy.js:432` had this right all along, so the **56-feature vector was never affected**. Corrected, on 63,305 usable turns, **nine of eighteen pair weights flip sign:** | pair term | old | refit + matcher fix | ---|---|---| | `spreadFreeBesideAlly` | -4.986 | **+0.863** | | `terrainSetupHelpsPartner` | -4.125 | **+2.005** | | `screenWhileThreatened` | -2.982 | **+0.110** | | `speedSetupHelpsPartner` | -1.595 | **+0.540** | | `weatherSetupHelpsPartner` | -0.549 | **+1.292** | Every "one slot sets up, the partner benefits" term was negative and is now positive. Held out, the joint layer takes top-1 from 9.7% to 11.5%. **And it wins.** Refit+fix against the shipped vector, seed-paired, greedy and joint on both arms, on three disjoint seed blocks: | run | games | decisive pairs to the refit | SPRT decided at | ---|---|---|---| | first | 17,350 | 66.7% | 78 decisive pairs | | replication | 1,934 | 65.9% | 69 | | challenger in arm 1, as convention | 1,932 | **70.9%** [66.0, 75.4] | 71 | The third is the one to quote: `sprt.js` and `paired_h2h.js` were run over it independently and agree to the decimal. All three are far past the 55% ship threshold, and every one was decided inside ~420 games — the fixed 200,000-game run this started as was 99.8% waste. **Item 1's stated premise is void.** It reads -5.054 as evidence that the imitation objective prices good play badly.

That number was a fitter defect. Item 1 may still be worth doing — the objective argument has independent support from greedy — but **this is no longer evidence for it**, and DODUO's 42.0% was measured with the contaminated vector and no longer describes the current one. **A correction about the analysers, recorded because it was nearly shipped as a fix.** The first reading of this H2H looked inverted, and `sprt.js` was changed to treat arm 2 as the challenger on the strength of `mew.js:215` calling `--weights2` "the challenger". That comment is about the EXPLOITABILITY search, not the standard A/B. **Arm 1 is the challenger** — `paired_h2h.js:183` builds its NEW label from arm 1, and the run that measured greedy at 79.7% put `--greedy` on arm 1. The change was reverted; making it would have put the two analysers at odds and inverted every run already analysed. What WAS wrong is narrower: `sprt`'s name-based fallback hardcoded 'score' as the new arm, so on `--policy prior --policy2 score` it disagreed with its own arm-based path. Fixed, and `tests/test-sprt-arm-sign.js` now asserts the two analysers use the same rule so they cannot drift.

1. DODUO trained for WINNING, not for resemblance. The most precise open question in the project. DODUO is fully built: `fit_joint.js` fits the pair block, `magnemite.js` plays it (`--joint`), `--joint-zero` is a true control running the whole pair path with only the coordination weights zeroed, top-K is capped, and fallbacks are counted in `stats.jointFellBack` . It lost at 42.0%, and the fit itself says why — it prices `spreadFreeBesideAlly` at **-5.054**, which means *humans rarely click this*, not *this is bad*. A bot told to avoid a free spread move beside its own ally by -5 will decline its best plays.

2026-07-30 established the pattern that makes this item one: **four knowledge additions produced four nulls; two changes to the OBJECTIVE produced two large wins** (greedy +12 points / 79.7% of decisive pairs; self-play 55.9%). The objective is the binding constraint, and DODUO has only ever been fitted to the losing one.

RECONCILED 2026-07-31. That 55.9% was measured on the **53-feature vector with switching OFF**. Repeating the experiment on the **56-feature vector with switching ON** gives **48.1%** [46.5, 49.8] over 9,728 paired games — a interval entirely below 50, i.e. self-play training made the policy worse. Both numbers stand as measurements of different configurations; neither generalises to 'self-play helps'. The difference is not explained, and three candidate causes are untested: switching exploration being harmful (consistent with the older 10-point switching loss), 36.5% drift over 18 iterations, or self-play eroding imitation-fitted features that were already good.

THE WIRING IS DONE, 2026-07-31 (commit `ce5367c`). It was wiring, as this item predicted, and the arithmetic in the sentence below was wrong in its parts while right in its total: the vector is **56 singles + 18 pair terms = 74**, not 53 + 21. The pair softmax is $P(q) = \exp(s_q) / \sum \exp(s_k)$ with $s_q = w_S \cdot x_a + w_S \cdot x_b + w_J \cdot j_f$, so the feature vector attached to pair q is the concatenation $[x_a + x_b, j_f]$ — the two single vectors **summed**, because both are scored by the same single block. `accumulateLogitGrad` was already generic over length, so no new mathematics was needed. **Four separate faults would each have produced a plausible and wrong learning curve**, and every one was caught by a check rather than by reading the code: | fault | what it would have looked like | |---|---| | `mew.js` sized the run gradient from the **single** weight file and summed with `k < GRAD.length` | 74 truncated to 56 — every pair weight silently dropped, curve printed as normal | | `magnemite.js` allocated `learnGrad` ~50 lines **before** `this.wj` loaded, so `this.joint` was still false | every joint player returns 56 entries; caught by the above on the first 6-game probe | | `magnemite.js` read the joint vector from a **hardcoded** path | every iteration replays the frozen shipped fit; pair terms never move; flat curve reads as convergence | | `preflight.js` indexed only `B.FEATURES` | the gate certifies a joint run **without checking the block that run exists to train** | `--joint` on one arm is the A/B experiment, not a training configuration. `mew.js` now refuses `--learn` with the layer on one side (the run gradient sums both sides, and index k is a different feature in each); `train_policy.js` adds the mirror flag rather than leaving it to be

remembered. **Measured:** preflight at 24 games with `--joint --joint2 --switching --switching2` — every block live, pair block 15 of 18 live, `|gradient|` 9.12. Two real iterations at 40 games each move the pair terms on their own merit: `bothSameTarget` **+0.164** (third-largest change in the whole 74-vector), `overkill` **+0.120**, `focusFireKills` **+0.094**. Suite 40 passed / 2 failed, the same two known failures.

TRAINED 2026-07-31 (evening). 12 iterations x 5,000 games with `--joint-only`, which freezes the 56 singles so the whole step reaches the pair block (the first run, unfrozen, moved it 4.7% -- see 2.8). Pair block moved **40.0%** of its own length; singles moved **0.000**, verified, so the head-to-head differs in the 18 coordination weights and nothing else. `|pair term| resemble | win | |---|---|---|---|` `bothSameTarget` `| 0.031 | 2.249 |` `overkill` `| -0.951 | +0.980 | SIGN FLIP |` `focusFireKills` `| -0.107 | +1.094 | SIGN FLIP |` `partnerCoversMe` `| -0.004 | +0.639 | SIGN FLIP |` `spreadFreeBesideAlly` `| -4.986 | -4.815 |` barely moved | Three of the four largest changes are sign flips and they tell one story: **the imitation fit penalised focus fire, and training to win reverses it**. Humans spread damage; winning concentrates it. `spreadFreeBesideAlly` **is the exception and it is this item own headline weight**. The argument above is that a bot told to avoid a free spread move beside its ally by `-5` will decline its best plays. After training it is `-4.8`. Whatever moved the other three did not move this one, and that is unexplained. **WHY IT DID NOT MOVE — ANSWERED 2026-07-31 (Will).** Not cancellation and not a dead feature: the gradient was **positive in all 12 iterations, never once negative**, so the evidence is unanimous that `-4.99` is too negative. It is $\sim 13\times$ smaller per round than `bothSameTarget` because the feature `UNDER-FIRES`, and the reason is a detector that is too narrow: ``js const freeSpread = (c, x) => c && c.move && c.spread && c.spread.length > 1 && F(x, 'allyHit') === 0; `` `allyHit` is a SINGLE-SLOT feature computed from the partner's TYPE and ABILITY — Levitate, Flash Fire, Volt Absorb, Earth Eater, or resisting the type. ****A partner who clicks PROTECT is spared exactly as completely as a Flying partner, and the feature cannot see it****, because Protect is not a property of the partner; it is a choice the partner makes that turn. That is structural, and it is the whole argument for the pair layer: `spreadFreeBesideAlly` RECEIVES both candidates and reads the `allyHit` of its own slot only. The partner's Protect is already in hand and is never consulted.

****Measured cost:**** Earthquake is on ****69 of 308 species (22.4%)***, by far the commonest spread move, and it is `allAdjacent` — so today it qualifies only when the partner is immune. 40 species (13.0%) carry an `allAdjacentFoes` move that spares the ally automatically; 83 (26.9%) carry an `allAdjacent` move that does not. The commonest real line in VGC — Earthquake plus partner Protect — is in the second group and is invisible. ****FIX, QUEUED** (`board.js` was in use by the head-to-head, per the no-edit-during-a-run rule): ****extend `freeSpread` to treat the partner candidate as sparing the ally when it is a protect-family move. `PROTECTMOVES` and `tgtMayProtect` already exist, so nothing is typed in. `WIDE GUARD: RESOLVED`, and it is BOTH cases depending on whose side it is on.**** On MY side it spares my partner from my own spread move exactly as Protect does. On the FOE side it blocks my spread move outright, which is an anti-synergy and belongs in the dead-move tests, not here.

****THE FULL SET IS DERIVED, NOT TYPED**** — probe each candidate move's volatile/side condition by calling its real `onTryHit` against the actual spread move, the same technique `entryEffects` and `speedStub` already use. Measured 2026-07-31: `| spares the ally (10) | excluded, and why | |---|---|` `| Protect, Detect, Spiky Shield, Baneful Bunker, Burning Bulwark, Silk Trap, King's Shield, Obstruct, Wide Guard, Mat Block |` ****Quick Guard**** — priority only; ****Crafty Shield**** — status only; ****Endure**** — no `onTryHit` at all, it prevents FAINTING, not damage | Three flag-based shortcuts were tried and are all WRONG, recorded so they are not retried: `stallingMove` admits ****Endure****; the presence of `onTryHit` admits ****Quick Guard** and ****Crafty Shield****; `target === 'allySide'` admits both of those and misses every self-targeting Protect. Only running the handler against the move separates them. ****This weakens the interpretation of the head-to-head now running.**** Both arms share the same detector, so the comparison is still fair, but neither arm can express the coordination pattern the feature is named for. ****MEASURED 2026-07-31. IT LOST.**** 194,514 paired games, 97,257

complete pairs. | | | |---|---| | decisive pairs | ****26,405**** (27.1% of all pairs) | | NEW (trained to win) takes | ****45.7%****, 95% CI ****[45.1, 46.3]**** | | verdict | ****worse**, and the interval clears 50** | Training the 18 coordination weights on the game result made the policy WORSE than the same weights fitted to imitate a human click. The three sign flips that looked like a coherent story about focus fire do not pay. ****The likely reason is in the same table: 72.9% of pairs were 1-1 splits****, i.e. the TEAM decided the game and not the policy. That contamination is in the TRAINING gradient too, so REINFORCE on win/loss was mostly fitting a team lottery. The objective was right; the ESTIMATOR of it was not. Three fixes follow, and none of them is "go back to imitation": 1. ****Pair the TRAINING runs**** as the tests already are, so team luck cancels out of the gradient. 2. ****Score decisions against a win-probability baseline**** (PORY) rather than the final result, so a move is credited with what it changed instead of what happened six turns later. 3. ****Train against a pool, not only against self**** – a self-play policy beats its training partner, and this one had never faced the imitation policy until the head-to-head it lost. That is VGC-Bench's own finding, already recorded in docs/COMPETITORS.md. ****CAVEAT, AND IT IS LARGE.**** The whole-repo review found `engine/fit_policy.js:170` computes the reweighting factors from ONE store and applies them across all three, so the shipped weight vector was fitted against the wrong population. Both arms here share those singles, so this comparison is internally fair – but every ABSOLUTE number in this project is provisional until that refit. ****engine/sprt.js** reached this verdict after 3,516 games instead of 194,514 – 98.2% of the run saved. ****--verify** confirms it agrees with `paired_h2h.js` exactly (12,073 and 14,332 on both readers). A 136-minute run becomes about two and a half minutes. What remains: ``node engine/train_policy.js --joint --switching --switching2 --procs 6 --iters <N> --games <G>`` then the head-to-head the trainer now prints, in which ****both arms run the pair path with identical singles and only the 18 pair weights differ**** – so it measures **trained to win** against **fitted to resemble**, and nothing else. The previous printout passed a 74-length file to `--weights``, which *would have been rejected outright or, worse, run an experiment whose arms differed in the wrong block.*

A trap already paid for once: `fit_joint` fits its single block and its pair block TOGETHER, and 23 of 48 features carry opposite signs between that fit and the shipped one. Mixing the two vectors lost 31.2% on decisive pairs and would have been reported as "coordination does not help."

2. Re-run WOBBUFFET against the current vector. (Will, 2026-07-30: put this back on the list.) **ATTEMPTED 2026-08-04 AND VOID – still open, and now there is no number at all rather than a stale one.** ~Its result is still the most important number in the repo and it is three feature-generations stale: a counter found in forty minutes beat MAG **63.2%** [56.6, 69.3] on the 17-feature vector, mirror control 47.5%.~ The re-run's defender was refitted mid-run and the simulator moved twice more; separately its hill-climb accepted 1 of 24 steps, so `exploit.js` needs a stamp and a dimension-aware step rule before the next attempt. `docs/SEARCH.md` §R8 has the prepared command and its three preconditions. That challenger was not rock-paper-scissors – it was a better player drawn from the same features and optimised for wins rather than resemblance, which is item 1's lesson in another form. `engine/exploit.js --target <weights.json>` can now be pointed at DODUO, so the EXPLOITABILITY argument for coordination – the one the 42.0% result explicitly does not settle – can finally be tested instead of argued.

3. MACHAMP, re-run on the current vector. (Will, 2026-07-30: keep it.) Half-run and stale: the 2026-07-26 run completed 2 of 6 generations on a **17-feature** vector and recorded no verdict; the vector is now 53. Its METHOD is alive – champion/challenger promotion behind a Wilson interval is the same win-objective idea `train_policy.js` now implements by policy gradient. Keep the guard that made it honest: every promoted champion plays EVERY previous generation, so that "gen 5 beats gen 4" is not read as progress by itself. Kept as insurance rather than as evidence – the "this metagame is cyclic" justification is withdrawn (2026-08-02); see `docs/MODELS.md` under MACHAMP.

4. Branch scoring with a mixed strategy — the research is done, so this is no longer vague. Measured on a real mid-game board: $9 \times 8 = 72$ **joint actions per side**, $72 \times 72 = 5,184$ **matrix cells**. The 10^{12} figure from VGC-Bench is the game TREE, not one turn.

- Only **28 of 72** pairs have a non-zero joint vector. The other **44 are free** — their score is exactly the sum of two singles already computed.
- With two Pokémon the coordination graph is a SINGLE EDGE, so Variable Elimination reduces to enumerating the 72 pairs and Max-Plus is unnecessary. The MARL factorisation literature (QMIX, QPLEX, Weighted QMIX) exists to avoid enumeration at many agents, which is not our problem.
- What that literature DOES tell us: QMIX's monotonicity constraint cannot represent non-monotonic coordination — exactly "Protect while my partner removes the threat." Independent per-slot scoring cannot express it either. Not "scores it badly": cannot express it.
- Pruning order matters. Structural prunes (choice lock, spread moves with no target, immunities) are exact and happen BEFORE scoring. Score-based top-K is circular for the 72 and only pays for the 5,184.
- Pokémon is SIMULTANEOUS-move, so the answer at a node is a mixed strategy, not an argmax. Minimax is the wrong algorithm; this is poker-shaped. SLOWKING's regret matching already does this mathematics at team preview and nobody has pointed it at a turn.

5. A leaf evaluator, so depth > 1 is affordable. `position_features.js` (16 features) plus PORY retrained on clean data. Without it every node rolls to the end; with it, depth 2 is 625 cheap evaluations.

6. Retire the second scorer in `app/index.html`. (Now the only failing test in the suite, and the only one at the `v3.31.0` tag.) It assigns **21 of 56 features** — 35 are never written and silently read 0, and 8 of the 21 disagree with the engine outright. This is the "facts are global" rule broken inside the UI.

`app/scoreboard.html` already does it correctly by rendering engine-computed values. The real fix is to ship the actual engine to the browser — `medicham2-browser.js` already runs in both, and `board.js` has no top-level requires; the only blocker is one `process.env` read at module scope.

7. Bring the five living docs current, and keep them current. Whitepaper, deck, technical docs, SUMMARY and MODELS drifted to 3.21.0 against a CHANGELOG at 3.28.0 once already — see the process failure recorded in 2.6. SUMMARY, MODELS, the white paper, PUBLICATION and this file were brought to 3.31.0 on 2026-07-31; the deck and technical docs have NOT been checked since.

8. Measure exploitability again, against the mixed-strategy version. The test that says whether the project has moved from *plays okay* onto the branch that can approach *solved*.

9. External validation. Our bots grading each other cannot establish competence. The two real scoreboards are the Bo3 open-sheet ladder — where the hidden-information problem does not exist and we already collect the games — and VGC-Bench's published agent results on that same format.

What is genuinely strong: the data discipline, the validated engine, the measurement culture, and the tooling that now enforces all three. The project's most valuable output to date is not a model — it is the ability to tell which of its own results are real, which was demonstrated by dissolving four of them in a single day.

What is not: every in-battle model. MAG is a competent starting position and a measurably exploitable one. No model in this project has yet been shown to play VGC well against anything other than a worse version of itself.

The gap between those two paragraphs is the whole roadmap.